Übungsblatt 2: RISC-V-Registerbefehle

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Das Korrekturprogramm von Hand ausführen (8 Punkte)

Ein Drucksensor liefert ganzzahlige Rohwerte. Ein kurzes RV32I-Programm korrigiert sie nach $Y = (A - B) + 4C$: A ist der Rohwert, B der Nullpunkt des Sensors und C eine Temperaturstufe, die je Stufe vier Zählwerte ausgleicht. Das Programm begleitet Sie durch das ganze Blatt. Es beginnt nach Lehrannahme bei Adresse `0x1000`; ohne komprimierte Befehle ist jeder native Befehl vier Byte lang.

```
addi t0, x0, 12       # A: Rohwert
addi t1, x0, 19       # B: Nullpunkt
addi t2, x0, 5        # C: Temperaturstufe
sub  t3, t0, t1       # A - B
slli t4, t2, 2        # 4C durch Schieben
add  t5, t3, t4       # Y
```

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (5 P)

Adresse	Befehl	gelesene Werte	Ergebnis	Wirkung
`0x1000`	`addi t0, x0, 12`	0 und Konstante 12	12	t0 = 12
	`addi t1, x0, 19`			
	`addi t2, x0, 5`			
	`sub t3, t0, t1`			
	`slli t4, t2, 2`			
	`add t5, t3, t4`			

2. Geben Sie t3 als 32-Bit-Hexwort an. Unter welchen Architekturnamen `x...` finden Sie t3 und t5 in einer Registeranzeige? (1 P)
3. Welchen Wert hat der PC nach dem letzten Befehl, und wie viele Byte belegt das Programm? (1 P)
4. Um ein Register zu sparen, schreibt jemand in Zeile 4 `sub t0, t0, t1` und in Zeile 6 `add t5, t0, t4`. Welchen Wert hat t0 danach? Stimmt Y noch, und welche Eingabe ist anschließend verloren? (1 P)

Musterlösung

Adresse	Befehl	gelesene Werte	Ergebnis	Wirkung
`0x1004`	`addi t1, x0, 19`	0 und Konstante 19	19	t1 = 19
`0x1008`	`addi t2, x0, 5`	0 und Konstante 5	5	t2 = 5
`0x100C`	`sub t3, t0, t1`	12 und 19	−7	t3 = −7
`0x1010`	`slli t4, t2, 2`	5, Weite 2	20	t4 = 20
`0x1014`	`add t5, t3, t4`	−7 und 20	13	t5 = 13

1. Je Zeile 1 P. Die Eingaben t0 bis t2 bleiben erhalten, weil sie nur als Quellen vorkommen.
2. t3 = `0xFFFFFFF9`, also −7. t3 ist x28, t5 ist x30 (1 P).
3. PC = `0x1018`. Sechs Befehle zu je vier Byte belegen 24 Byte (1 P).
4. sub liest den alten Wert 12 und schreibt $12 - 19 = -7$ nach t0. Y $= -7 + 20 = 13$ stimmt weiterhin, aber der Rohwert $A = 12$ ist überschrieben und für spätere Rechnungen verloren (1 P).

Aufgabe 2: Statuswort und Messzähler (10 Punkte)

Neben dem Messwert liefert der Sensor ein Statuswort, hier t0 = `0x0000004D`; die unteren acht Bit sind $0100\,1101_2$. Bit 0 meldet „Messwert gültig“, Bit 1 „Überlast“ und Bit 7 „Selbsttest läuft“. Logische Immediates werden wie bei addi von zwölf auf 32 Bit vorzeichenerweitert.

1. Ergänzen Sie Maske und Ergebnis. Jede Zeile startet mit t0 = `0x0000004D`. Die erste Zeile ist ein Beispiel. Tipp: Die Maske ist das vorzeichenerweiterte Immediate. (5 P)

Befehl	Zweck	Maske (32 Bit)	Ergebnis
`andi t1, t0, 0x0F`	untere vier Bit behalten	`0x0000000F`	`0x0000000D`
`andi t2, t0, 1`	Bit 0 prüfen		
`ori t3, t0, 0x80`	Bit 7 setzen		
`xori t4, t0, 2`	Bit 1 kippen		
`andi t5, t0, -16`	untere vier Bit löschen		
`xori t6, t0, -1`	alle Bit invertieren		

2. Die unteren zwölf Bit des Statusworts sollen erhalten bleiben. Warum ist `andi t1, t0, 4095` keine gültige Instruktion? Geben Sie zwei Befehle an, die die Aufgabe lösen. (2 P)

Ein 32-Bit-Register zählt die Messungen; jede Messung führt `addi t0, t0, 1` aus.

3. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (2 P)

Zählerstand vorher	neues Wort	signed gelesen	vorzeichenlos gelesen
`0x00000009`	`0x0000000A`	10	10
`0x7FFFFFFF`			
`0xFFFFFFFF`			

4. Nach 2 147 483 647 Messungen zeigt die Anzeige plötzlich −2 147 483 648. Hat der Prozessor einen Fehler gemeldet? Welche Lesart hätte die Anzeige verwenden müssen? (1 P)

Musterlösung

Befehl	Zweck	Maske (32 Bit)	Ergebnis
`andi t2, t0, 1`	Bit 0 prüfen	`0x00000001`	`0x00000001`
`ori t3, t0, 0x80`	Bit 7 setzen	`0x00000080`	`0x000000CD`
`xori t4, t0, 2`	Bit 1 kippen	`0x00000002`	`0x0000004F`
`andi t5, t0, -16`	untere vier Bit löschen	`0xFFFFFFF0`	`0x00000040`
`xori t6, t0, -1`	alle Bit invertieren	`0xFFFFFFFF`	`0xFFFFFFB2`

1. Je Zeile 1 P für Maske und Ergebnis. Bit 0 ist gesetzt, der Messwert ist gültig. Bei −16 und −1 füllt die Vorzeichenerweiterung die Maske oben mit Einsen; xori mit −1 invertiert deshalb alle 32 Bit.

2. 4095 liegt außerhalb des signed Zwölf-Bit-Bereichs −2048 bis 2047 (1 P). Zum Beispiel `li t6, 4095` und `and t1, t0, t6`: Die positive Maske steht dann im Register (1 P).

Zählerstand vorher	neues Wort	signed gelesen	vorzeichenlos gelesen
`0x7FFFFFFF`	`0x80000000`	−2 147 483 648	2 147 483 648
`0xFFFFFFFF`	`0x00000000`	0	0 (Umlauf)

3. Je Zeile 1 P.
4. Nein. addi speichert die unteren 32 Bit; ein signed Überlauf löst in RV32I keinen Trap aus und setzt kein Überlaufflag. Vorzeichenlos gelesen ist `0x80000000` der richtige Zählerstand 2 147 483 648 (1 P).

Aufgabe 3: Mit Schieben multiplizieren und mitteln (8 Punkte)

Zeile 5 des Korrekturprogramms bildet $4C$ durch Linksschieben, denn Multiplikationsbefehle gehören zur Erweiterung M, nicht zu RV32I. Außerdem mittelt das Gerät vier korrigierte Werte: Ihre Summe steht in t6, und `srai t6, t6, 2` teilt sie durch vier.

1. Ergänzen Sie die Tabelle. Jede Zeile startet unabhängig. Die erste Zeile ist ein Beispiel. (4 P)

Befehl	Quellen vorher	Ergebnis (hex)	dezimal
`slli t4, t2, 2`	t2 = 5	`0x00000014`	20
`slli t4, t2, 2`	t2 = −2 = `0xFFFFFFFE`		
`srai t6, t6, 2`	t6 = 52 = `0x00000034`		
`srai t6, t6, 2`	t6 = −28 = `0xFFFFFFE4`		
`sll t4, t2, t5`	t2 = 5, t5 = 34		

2. Ein anderes Gerät zeigt bei der Summe −28 den Mittelwert 1 073 741 817 an; bei positiven Summen stimmt der Wert. Welcher Befehl steht dort vermutlich statt `srai`? Begründen Sie mit dem Bitmuster. (2 P)
3. Die vier Werte −7, −8, −7 und −8 ergeben die Summe −30 = `0xFFFFFFE2`. Welchen Wert liefert `srai t6, t6, 2`? Wie liegt er zum mathematischen Mittelwert −7,5? (1 P)
4. Mit t2 = 5 und t5 = 32 liefert `sll t4, t2, t5` den Wert 5. Warum? (1 P)

Musterlösung

Befehl	Quellen vorher	Ergebnis (hex)	dezimal
`slli t4, t2, 2`	t2 = −2 = `0xFFFFFFFE`	`0xFFFFFFF8`	−8
`srai t6, t6, 2`	t6 = 52 = `0x00000034`	`0x0000000D`	13
`srai t6, t6, 2`	t6 = −28 = `0xFFFFFFE4`	`0xFFFFFFF9`	−7
`sll t4, t2, t5`	t2 = 5, t5 = 34	`0x00000014`	20

1. Je Zeile 1 P. Linksschieben um zwei Stellen vervierfacht auch negative Werte, solange keine relevante Stelle herausfällt. Bei sll zählen nur die unteren fünf Bit von t5: $34 \bmod 32 = 2$.
2. `srli t6, t6, 2` (1 P). srli füllt links mit Nullen: Aus `0xFFFFFFE4` wird `0x3FFFFFF9` = 1 073 741 817. srai kopiert das Vorzeichenbit und liefert `0xFFFFFFF9` = −7 (1 P).
3. `0xFFFFFFF8` = −8. srai rundet nach unten, nicht zur Null; Abschneiden ergäbe −7 (1 P).
4. Nur die unteren fünf Bit von t5 zählen. 32 hat dort lauter Nullen, die effektive Weite ist 0 (1 P).

Aufgabe 4: Konstanten im Befehl (8 Punkte)

Die Zeilen 1 bis 3 des Korrekturprogramms setzen die Eingaben mit addi. Das Immediate-Feld ist zwölf Bit breit und signed: Es reicht von −2048 bis 2047. Eine direkt angegebene Shiftweite reicht von 0 bis 31.

1. Ist die Zeile eine native Instruktion, eine Pseudoinstruktion oder unzulässig? Begründen Sie kurz. Die erste Zeile ist ein Beispiel. (3 P)

Zeile	Art	Begründung
`addi t1, x0, 19`	nativ	19 liegt im Bereich −2048 bis 2047
`addi t1, x0, -2048`		
`addi t1, x0, 2048`		
`addi t3, t0, -19`		
`li t1, 3000`		
`mv t3, t0`		
`slli t4, t2, 32`		

2. Weil der Nullpunkt fest 19 ist, kann `addi t3, t0, -19` die Zeilen 2 und 4 ersetzen. Geben Sie −19 als Zwölf-Bit-Muster, als drei Hexziffern und als vorzeichenerweitertes 32-Bit-Wort an. (1 P)
3. Für welche der Konstanten $K = 19$, $K = 2048$ und $K = -2048$ lässt sich t0 $- K$ mit einem einzigen addi berechnen? (1 P)
4. Ein neuer Sensor hat den Nullpunkt $B = 3000$. Eine gültige native Entwicklung von `li t1, 3000` ist `lui t1, 0x1` gefolgt von `addi t1, t1, k`. Welchen Wert hat t1 nach lui, und welches k ist nötig? Tipp: lui setzt die Bits 31 bis 12 und löscht die unteren zwölf Bit. (2 P)
5. Zeile 2 des Programms aus Aufgabe 1 lautet nun `li t1, 3000`. Wie viele Byte belegt das Programm jetzt, und an welcher Adresse steht sub? (1 P)

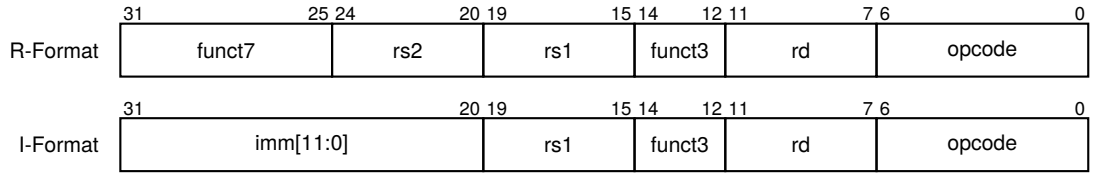
Musterlösung

Zeile	Art	Begründung
`addi t1, x0, -2048`	nativ	untere Grenze des Bereichs
`addi t1, x0, 2048`	unzulässig	größer als 2047
`addi t3, t0, -19`	nativ	−19 liegt im Bereich, zieht 19 ab
`li t1, 3000`	Pseudo	Assembler erzeugt mehrere native Befehle
`mv t3, t0`	Pseudo	native Entwicklung `addi t3, t0, 0`
`slli t4, t2, 32`	unzulässig	direkte Weite nur 0 bis 31

1. Je Zeile 0,5 P.
2. `1111 1110 1101` = `0xFED`, erweitert `0xFFFFFFED` (1 P). Kontrolle: $4077 - 4096 = -19$.
3. $K = 19$ mit Immediate −19 und $K = 2048$ mit Immediate −2048. Für $K = -2048$ wäre +2048 nötig, das außerhalb des Bereichs liegt (1 P).
4. Nach lui: `0x00001000` = 4096 (1 P). $k = 3000 - 4096 = -1096$; das liegt im Bereich (1 P).
5. Sieben native Befehle, also 28 Byte. sub steht jetzt bei `0x1010` (1 P).

Aufgabe 5: Das Korrekturprogramm codieren (14 Punkte)

Die Felder stehen von Bit 31 links bis Bit 0 rechts. Registerfelder enthalten Nummern: x0 = 0, t0 bis t2 = x5 bis x7, t3 bis t6 = x28 bis x31.



Befehl	opcode	funct3	funct7
add	`0110011`	`000`	`0000000`
sub	`0110011`	`000`	`0100000`
addi	`0010011`	`000`	—

1. Ergänzen Sie die Felder von `sub t3, t0, t1`. Die erste Zeile ist ein Beispiel. (2 P)

Befehl	funct7	rs2	rs1	funct3	rd	opcode
`add t5, t3, t4`	`0000000`	`11101`	`11100`	`000`	`11110`	`0110011`
`sub t3, t0, t1`						

2. Schreiben Sie das Wort von `sub t3, t0, t1` in Vierergruppen und als Hexwort. Zum Vergleich ergibt das Beispiel `0000 0001 1101 1110 0000 1111 0011 0011` = `0x01DE0F33`. (2 P)
3. add und sub haben denselben opcode. Welches Feld unterscheidet sie, und welches Instruktionsbit ändert sich dabei? (1 P)
4. Ergänzen Sie die Felder des I-Formats und das Hexwort. Die erste Spalte ist ein Beispiel. (4 P)

Feld	addi t0, x0, 12	addi t1, x0, 19	addi t3, t0, −19
imm[11:0]	`000000001100`		
rs1	`00000`		
funct3	`000`		
rd	`00101`		
opcode	`0010011`		
Hexwort	`0x00C00293`		

5. Lesen Sie aus Ihrem Hexwort für `addi t3, t0, -19` die oberen drei Hexziffern als signed Zwölf-Bit-Zahl. Erhalten Sie −19 zurück? (1 P)
6. Für eine neue Messung ist der Rohwert $A = 30$. Welche Wörter des Programms aus Aufgabe 1 ändern sich? Warum bleibt das Wort von sub gleich? (2 P)
7. Das Wort von `sub t3, t0, t1` liegt ab Adresse `0x100C`. Ergänzen Sie die Bytes in Little Endian. (2 P)

Adresse	0x100C	0x100D	0x100E	0x100F
Byte				

Musterlösung

Befehl	funct7	rs2	rs1	funct3	rd	opcode
sub t3, t0, t1	0100000	00110	00101	000	11100	0110011

1. rs2 = 6 (t1), rs1 = 5 (t0), rd = 28 (t3). Felder 2 P, anteilig.
2. 0100 0000 0110 0010 1000 1110 0011 0011 = 0x40628E33. Vierergruppen 1 P, Hexwort 1 P.
3. funct7: 0000000 bei add, 0100000 bei sub. Es ändert sich nur Instruktionsbit 30 (1 P).

Feld	addi t1, x0, 19	addi t3, t0, −19
imm[11:0]	000000010011	111111101101
rs1	00000	00101
funct3	000	000
rd	00110	11100
opcode	0010011	0010011
Hexwort	0x01300313	0xFED28E13

4. Je Spalte Felder 1 P, Hexwort 1 P.
5. 0xFED = 4077; signed $4077 - 4096 = -19$ (1 P).
6. Nur das Wort der ersten Zeile: addi t0, x0, 30 = 0x01E00293 (1 P). sub codiert die Registernummern 5, 6 und 28, nicht deren Inhalte (1 P).

Adresse	0x100C	0x100D	0x100E	0x100F
Byte	33	8E	62	40

7. Je Byte 0,5 P. Das niederwertigste Byte liegt an der kleinsten Adresse.

Aufgabe 6: Fehlersuche bei falschen Korrekturwerten (12 Punkte)

Drei Geräte sollen das Programm aus Aufgabe 1 mit $A = 12$, $B = 19$ und $C = 5$ ausführen. Richtig wäre $Y = 13$.

Gerät 1 meldet $Y = 27$.

1. Angenommen, t4 = 20 stimmt. Welcher Wert stand dann in t3? Wie kann er aus 12 und 19 entstanden sein? (2 P)
2. Ein Speicherabzug zeigt an den Adressen `0x100C` bis `0x100F` die Bytes `33`, `0E`, `53`, `40`. Setzen Sie das 32-Bit-Wort zusammen. (1 P)
3. Zerlegen Sie das Wort in die Felder des R-Formats und geben Sie die Registernummern an. (3 P)

funct7	rs2	rs1	funct3	rd	opcode

4. Welcher Befehl steht dort, geschrieben mit Aliasnamen? Bestätigt das Ihre Vermutung aus Teil 1? Welche Bytes müssten ab `0x100C` stehen? (2 P)

Gerät 2 meldet $Y = 33$.

5. Angenommen, t3 = −7 stimmt. Welcher Wert stand in t4? Welcher Befehl wurde vermutlich falsch eingegeben, und wie lautet er? (2 P)

Gerät 3 soll den neuen Sensor mit dem Nullpunkt 3000 bedienen. Jemand hat Zeile 2 als `addi t1, x0, 3000` geschrieben; die Übersetzung scheitert.

6. Warum? Wie schreiben Sie die Zeile, damit t1 = 3000 entsteht? (2 P)

Musterlösung

1. $27 - 20 = 7$ (1 P). $7 = 19 - 12$: Die Quellen der Subtraktion sind vertauscht, gerechnet wurde $B - A$ statt $A - B$ (1 P).
2. Little Endian: Das Byte an der kleinsten Adresse ist das niederwertigste, also `0x40530E33` (1 P).

funct7	rs2	rs1	funct3	rd	opcode
`0100000`	`00101` = 5	`00110` = 6	`000`	`11100` = 28	`0110011`

3. Je Feld 0,5 P.
4. opcode, funct3 und funct7 ergeben sub: `sub x28, x6, x5` = `sub t3, t1, t0` (1 P). Das bestätigt die vertauschten Quellen. Richtig wären die Bytes `33 8E 62 40` aus Aufgabe 5 (1 P).
5. $33 - (-7) = 40 = 8 \cdot 5$ (1 P). Geschoben wurde um drei statt um zwei Stellen: `slli t4, t2, 3` statt `slli t4, t2, 2` (1 P).
6. 3000 liegt außerhalb des Bereichs −2048 bis 2047 und passt nicht in das Immediate-Feld (1 P). Richtig ist `li t1, 3000` oder ausdrücklich `lui t1, 0x1` und `addi t1, t1, -1096` (1 P).

Aufgabe 7: Die Messkorrektur in Venus prüfen

Ergänzen Sie den folgenden Programmrahmen. `li` setzt die Eingaben; bei Werten von −2048 bis 2047 entsteht daraus ein einzelnes addi.

```
.text
.globl main
main:
    li   t0, 12          # A: Rohwert
    li   t1, 19          # B: Nullpunkt
    li   t2, 5           # C: Temperaturstufe
    # Lücke 1: drei Befehle, Ergebnis Y in t5
Ergebnis_pruefen:
    # Lücke 2: Y als Ganzzahl ausgeben
    # Lücke 3: Programm beenden
```

Venus-Dienste der Kursumgebung: a0 enthält die Dienstnummer, danach folgt `ecall`. Dienst 1 gibt die Ganzzahl in a1 aus, Dienst 10 beendet das Programm.

1. Füllen Sie die Lücken. Lücke 1 übernehmen Sie aus Aufgabe 1, für Lücke 2 und 3 genügen fünf Befehle.
2. Öffnen Sie Venus unter thaumicmekanism.github.io/venus und geben Sie das Programm im Tab *Editor* ein. Der Wechsel auf den Tab *Simulator* assembliert es. Die Programmansicht zeigt die Spalten *Machine Code* (Hexwort), *Basic Code* (native Befehle nach der Entwicklung) und *Original Code*. Notieren Sie die Wörter von `li t0, 12` und von sub und vergleichen Sie mit Aufgabe 5.
3. Ein Klick auf einen Befehl setzt einen Haltepunkt (rote Zeile). Setzen Sie ihn auf den ersten Befehl nach `Ergebnis_pruefen` und starten Sie mit *Run*. Alternativ führt *Step* den grün markierten Befehl aus, *Prev* nimmt den letzten Schritt zurück. Prüfen Sie in der Registeransicht t3 bis t5 (x28 bis x30). Werte erscheinen standardmäßig vorzeichenlos hexadezimal, die Anzeigeeinstellung *Decimal* zeigt sie signed dezimal.
4. Führen Sie das Programm zu Ende und notieren Sie die Konsolenausgabe.
5. Sagen Sie für jede Zeile t3, t4 und Y voraus. Ändern Sie dann die drei li-Zeilen, assemblieren Sie neu und prüfen Sie. *Reset* setzt den Simulator auf den Anfangszustand zurück.

A	B	C	t3	t4	Y
12	19	5	−7	20	13
0	0	0			
−8	7	2			
9	−4	−2			
3012	3000	5			

6. Wie viele native Befehle zeigt die Spalte *Basic Code* in der letzten Variante für die drei li-Zeilen? Vergleichen Sie mit Aufgabe 4. Ersetzen Sie testweise `li t1, 3000` durch `addi t1, x0, 3000` und notieren Sie die Meldung.

Protokollieren Sie jeweils Vorhersage, Registeranzeige am Haltepunkt und Konsolenausgabe.

Musterlösung

```
.text
.globl main
main:
    li   t0, 12          # A: Rohwert
    li   t1, 19          # B: Nullpunkt
    li   t2, 5           # C: Temperaturstufe
    sub  t3, t0, t1      # A - B
    slli t4, t2, 2       # 4C
    add  t5, t3, t4      # Y
Ergebnis_pruefen:
    addi a1, t5, 0       # Druckwert nach a1
    addi a0, x0, 1       # Dienst 1: Ganzzahl ausgeben
    ecall
    addi a0, x0, 10      # Dienst 10: beenden
    ecall
```

Erwartung am Haltepunkt: t3 = `0xFFFFFFF9` (−7), t4 = `0x00000014` (20), t5 = `0x0000000D` (13). Die Konsole zeigt 13. `li t0, 12` wird zu `addi t0, x0, 12` = `0x00C00293`, sub hat das Wort `0x40628E33`.

A	B	C	t3	t4	Y
0	0	0	0	0	0
−8	7	2	−15	8	−7
9	−4	−2	13	−8	5
3012	3000	5	12	20	32

Laut Venus-Quelltext entwickelt li Werte außerhalb von −2048 bis 2047 in lui und addi: `li t0, 3012` ergibt `lui t0, 1` und `addi t0, t0, −1084`, `li t1, 3000` ergibt `lui t1, 1` und `addi t1, t1, −1096`. Die drei li-Zeilen erzeugen also fünf native Befehle. `addi t1, x0, 3000` lehnt der Assembler ab, weil das Immediate außerhalb des Bereichs liegt. Das sind Modellvorhersagen; Anzeige und Meldungen der eigenen Venus-Version gehören ins Protokoll.